Link to github repository: https://github.com/rinakzm256/Assignment-4

Generated Code Analysis:

OOP Implementation Table:

Class	Main Attributes	Important Methods	Responsibility	OOP Principles Applied
Patient	patient_id, name, dob, phone, address	__init__()	Data holder for patient demographic information.	Encapsulation
VisitNote	note_id, patient_id, date_time, clinician_id, note_text, category	__init__()	Data holder for clinical visit notes.	Encapsulation
DatabaseManager	db_path	get_connection(), _init_db()	Manages SQLite connection creation and initial table creation.	Abstraction
PatientManager	db	generate_patient_id(), register_patient(), search_patient(), update_permitted_demographics()	Registering, searching, and updating patients.	Abstraction, Encapsulation
VisitManager	db, ai_client	request_ai_category(),	Attaching notes to patients and	Abstraction

		add_visit_note(), get_patient_history()	reading history.	
AIServiceClient	simulate_timeout, timeout_seconds	suggest_category()	Handles communication with external AI.	Abstraction
AuthService	None	is_admin(), is_clinical()	Validates user role permissions for system operations.	Abstraction

Encapsulation: Used to bundle data and related logic into separate classes. Encapsulation is very limited. Attributes such as Patient.patient_id are publicly accessible and mutable (bad).

Abstraction: SQL and AI integration are both hidden behind a layer of abstraction. They are both functions callable from the GUI.

Inheritance: Not present

Polymorphism: Not present

Mandatory Analysis: Immutable Patient ID

1. Patient class, self.patient_id attribute
2. Nothing enforces immutability in version 1. The attribute is assigned publicly.
3. Version 1 as generated by Google Gemini takes no lengths to enforce immutability
4. It can be modified via direct access, any other method, and even via the database
5. uuid.uuid4() in PatientManager.generate_patient_id() ensures unique patient hash IDs
6. Attempting to modify the patient ID directly, it just let me without any error of pushback. This shows a critical flaw in the mutability of patient ID.

Code and Implementation Decisions:

1. Requirements & Architecture Alignment: PRSApp._save_visit_note() & submit_registration() in main_gui.py: User authorization is tightly coupled to UI event handlers. Additionally, strict role separation prevents administrative staff from recording visit notes. Suggested moving permission checks out of GUI event handlers into AuthService and allow admin staff access to clinic staff permissions.
2. OOP Design: Patient.__init__() in patient.py: The patient_id attribute is defined as a public variable and is not immutable. Each Patient ID must remain permanent and immutable. Suggested enforcing immutability by storing the value in a private variable and implementing a getter method.
3. Programming Standards: DatabaseManager.get_connection() in persistence.py: Database connections are opened without adhering to context manager conventions or cleanup blocks. If an error occurs during execution, connections remain open. Suggested introducing context and cleanup blocks to prevent resource leaks.
4. Code Structure and Maintainability: PRSApp._handle_search() and PRSApp._save_visit_note() in main_gui.py: Direct manipulation of UI is all bundled into overly long GUI methods, and the GUI handles formatting data strings directly. Suggested to integrate helper methods to break up the overly long GUI methods, leaving the GUI methods responsible for the GUI solely.
5. Efficiency and Optimization: PatientManager.search_patient() in patient.py: The search method runs two separate database queries sequentially, searching both for patient ID and name. This creates excess overhead on each search. Suggested to combine both queries into one optimized query that performs both functions in one.

Code Review and Implementation Analysis

Requirement ID	Class/Module	Method	How it is implemented
FR-01 (Patient ID Generation)	PatientManager (patient.py)	generate_patient_id()	Generates a unique alphanumeric patient ID string P-********
FR-02 (Patient	PatientManager	register_patient(),	Accepts demographic

Registration)	(patient.py), PRSApp (main_gui.py)	submit_registration()	fields, instantiates a Patient object, and writes the record to SQLite using INSERT SQL.
FR-04 (Patient Search)	PatientManager (patient.py)	search_patient()	Accepts a search query string and executes an SQL query against the SQLite database.
FR-05 (Visit Note Creation)	VisitManager (visit.py)	add_visit_note()	Takes a valid patient_id, timestamp, clinician ID, and text body, instantiates a VisitNote, and saves it to the visit_notes database table linked by foreign key
NFR-03 (Role-Based Access Control)	AuthService (auth.py)	is_admin(), is_clinical()	Inspects the active user session role to grant or deny execution access to registration or clinical visit note creation

Patient.__init__() in patient.py:

- Responsibility: Creates a new Patient object and stores their personal details
- Relationship: It is used by PatientManager to convert database rows into Patient objects, and by main_gui.py to display patient info on the screen.
- Is the OOP Design Appropriate? No. It defines patient_id as a public variable. This allows any part of the program to change a patient's ID after it is created
- Retain: Demographic fields (name, dob, gender, contact_info)

- Correct: Change self.patient_id to a private variable, add a read-only getter
- Why: Ensures patient ID is immutable while still being able to be read.

DatabaseManager.get_connection() in persistence.py:

- Responsibility: Opens and returns a connection to the SQLite database file.
- Relationship: It is called by PatientManager and VisitManager whenever they need to save or search for data in the database.
- No. It returns raw database connections without using context or cleanup blocks. If an error happens while running a query, the connection stays open, leading to resource leaks and potentially damaged files.

```
def get_connection(self):
  conn = sqlite3.connect(self.db_path)
  conn.row_factory = sqlite3.Row
  return conn  # Returned raw without automated cleanup
```

- Keep: `conn.row_factory = sqlite3.Row` so query results can easily be read by column name.
- Correct: Update the method to use Python's `@contextmanager` decorator with a `try...finally` block. This ensures the database connection always closes automatically after every operation, even if an error occurs mid-query, preventing memory leaks and resource issues.

Compare Version 1 and Version 2

Class/Method	Problem in V1	Improvement in V2	Type of Change	Why it is better
Patient	Patient ID is publicly mutable	Encapsulated ID as a private attribute	Defect Correction	Patient is now ID is now immutable
PatientManager.search_patient	Sequentially executed separate SQL	Consolidated queries into a single SQL	Optimization	Reduces search from two queries to one

	queries for ID and name lookups	statement using WHERE patient_id = ? OR name LIKE ?		per search request
DatabaseManager.get_connection	Returned raw database connections without automated resource cleanup or exception recovery handling	Wrapped connection management in a context manager	Architectural Improvement	Protects against file corruption and resource leaks
AuthService	Role checks were hardcoded as direct string comparisons.	Created centralized, intent-based authorization methods	Architectural Improvement	Decouples security policy from GUI
PRSApp	Single methods combined permission evaluation, database queries, AI payload preparation, and UI text state updates	Decomposed handlers into focused helper methods	Programming-Standard Improvement	Improves maintainability and code readability by adhering to the Single Responsibility Principle

- The final application did not execute successfully, but after some prompting, I was able to get a working version.

- Yes, the identified problems were addressed, though it seemed to create new problems. I had to manually re-implement key features from V1 that the AI decided it just didn't want or need anymore.
- The build is a working prototype, but it runs on a local database with simulated AI logic rather than real AI integration. Additionally, the authorization is also simulated, as there is no database for staff.

Demonstrate Understanding

- Architecture to Implementation: The AI seemed to prioritize separation of concerns. It looked at my UML diagrams and directly translated those into modules, and it treated the requirements into each individual concern. The AI often implemented specific methods to ensure that specific requirements were met, though it missed some and needed to be reminded. It would seem that each issue was an individual issue, and it addressed them as such rather than focusing on the overall design.
- OOP Decisions: The AI used encapsulation via python's classes. It took advantage of classes wherever possible. Similarly, abstraction was used to cover up the API integrations, where all the program cares about is the output. The GUI itself is a high-level abstraction. Inheritance and polymorphism were used in the GUI. PRSApp inherits from Tkinter, which was the library that the AI chose to implement the UI via. In other places, polymorphism and inheritance would have been unnecessary, as I cannot, and apparently neither could the AI, think of a reason for it to be implemented.
- Patient ID Immutability: In the final implementation, patient ID is a private variable within the Patient class, and is read only.

```
def __init__(self, patient_id, name, dob, gender, contact_info,
address=None):
        self._patient_id = patient_id  # Private attribute for
immutability
        self.name = name
        self.dob = dob
        self.gender = gender
        self.contact_info = contact_info
        self.address = address

    @property
```

```
    def patient_id(self):
        """Read-only property enforcing immutable patient_id."""
        return self._patient_id
```

- Critical Code Analysis: The biggest problem was the patient mutability, but other problems came to light simply by attempting to use the basic functions of the programs. Because the code is so modular, I was able to trace potential problems with the encapsulation. As for the database vulnerabilities, in debugging mode, the database connections weren't being closed when errors were thrown.
- Refactoring and Optimization: The search optimization was a big one. It originally conducted 2 entire searches of the database, one for patient ID and one for name. Now, it is a singular query, which drastically improves overhead in theory. Additionally, making patient ID private solved one of the core issues of version 1. Now, patient IDs are protected from being changed after being declared.
- Code Understanding: main_gui.py acts as the central class for the entire program. It starts by implementing all of the other modules.

```
from ai_integration import AIServiceClient
from auth import AuthService
from patient import PatientManager
from persistence import DatabaseManager
from visit import VisitManager
```

The majority of main_gui focuses on the GUI itself, though in version 1, it implemented some logic that should have been left to the other modules. In version 2, it is almost entirely modular, relying on function calls from the other modules. When a user searches for a patient ID via the GUI, it calls search_patient(self, query) in patient.py. It then passes the query SELECT * FROM patients WHERE patient_id = ? OR name LIKE ? to SQLite, to which formatted data is returned. The GUI then takes this data that has been returned and displays it. The GUI itself is not the one doing the search.

```
def search_patient(self, query):
        """Combined single SQL query optimization for ID and name
matching."""
        with self.db.get_connection() as conn:
```

```
            cursor = conn.cursor()
            search_param = f"%{query}%"
            cursor.execute(
                """
                SELECT * FROM patients
                WHERE patient_id = ? OR name LIKE ?
                """,
                (query, search_param)
            )
            rows = cursor.fetchall()

        return [
            Patient(
                row["patient_id"],
                row["name"],
                row["dob"],
                row["gender"],
                row["contact_info"],
                row["address"]
            )
            for row in rows
        ]
```

GenAI Contribution:

As per this assignment, the bulk of the code was generated via Google Gemini, while I handled the verification steps. The engineering decisions were largely made by me, based on my schema and requirements, and I even instructed the model to ask further questions when choosing major decisions. I suggested improvements to be added in by genAi for version 2. From version 1 to version 2, the AI seemed to get confused and removed a lot of the functionality that was initially in version 2. For a large part, I went into version 1 and re-implemented a lot of version 1's code manually into version 2, though the AI assisted in updating the GUI. Overall, while Gemini definitely sped up the process, it used libraries I was unfamiliar with which made understanding difficult, and it made some mistakes that no reasonable person would ever have made.